

دليل المشروع الشامل ومرجع المناقشة الشفهية

Al-Razi University – Big Data Practical Midterm Project

Hybrid ELT Data Pipeline (Python Batch | Apache Spark | MongoDB)

المشرف الأكاديمي: م. عمر أبوسند

فهرس المحتويات

1. نظرة عامة وفلسفة المعمارية (#1) [ELT Philosophy & Architecture - نظرة عامة وفلسفة المعمارية]
2. دليل الملفات والدوال سطرًا بسطر (#2) [File-by-File & Function-by-Function Deep Dive - دليل الملفات والدوال سطرًا بسطر]
3. إقواعد التنظيف التسع والعزل وأثر التصحيح (9 #3) [Quality Rules & Quarantine - قواعد التنظيف-التسبع-والعزل-وأثر-التصحيح]
4. المسار المتقدم B: التحميل التزايدي (#4) [Track B: Incremental Delta Loading - المسار-المتقدم-b-التحميل-التزايدي]
5. سيناريوهات "ماذا لو؟" (#5) [What-If Analysis & Edge Cases - سيناريوهات-ماذا-لو؟]
6. بنك الأسئلة المتوقعة في المناقشة الشفهية مع الإجابات النموذجية (#6) [Doctor Viva Q&A - بنك-الأسئلة-المتوقعة-في-المناقشة-الشفهية]
7. دليل أوامر التشغيل ورفع المشروع إلى GitHub والتعامل مع ملفات جديدة [7# - دليل-أوامر-التشغيل-ورفع-المشروع]

1. نظرة عامة وفلسفة المعمارية

لماذا ELT وليس ETL؟

مما يؤدي إلى **فقدان البيانات غير الصالحة** واستحالة إعادة تحليل السجلات الخام في حال تغيرت قواعد العمل لاحقًا: ****ETL (Extract -> Transform -> Load) التقليدي**** - يقوم بتنظيف البيانات وفلترتها قبل تخزينها،

****نمطنا (ELT (Extract -> Load -> Transform)**** -

1. ****التحميل الأولي 100% الخام****: كل سجل يصل من ملف CSV يكتب أولاً فوراً كما هو في مجموعة `orders\_raw` دون أي حذف أو تعديل.

(`Audit Trail`)، وتصنيف السجلات إلى `VALID` أو `CORRECTED` لتذهب إلى `orders\_validated`، أو `QUARANTINE` لتذهب إلى `orders\_quarantine`.

2. ****التحويل والتصنيف اللاحق****: يتم تطبيق قواعد الجودة والتنظيف داخل الذاكرة مع حفظ مسار التدقيق

3. ****عدم فقدان أي معلومة****: تحقيق معادلة الاتساق الإلزامية:

$$\text{rows_read} = \text{valid_count} + \text{corrected_count} + \text{quarantine_count}$$

الموجه التلقائي للمحرك (Hybrid Engine Router):

يتميز بانعدام وقت إقلاع JVM واستخدام تقنية `Streaming` عبر `csv.DictReader` مع دفعات `bulk\_write`، مما يمنح استهلاك ذاكرة ثابت جداً وسرعة فائقة. -

****الملفات الصغيرة (\$\le 200\text{ MB}\$)****: يتم توجيهها تلقائياً إلى محرك ****Python Batch Loader****.

للاستفادة من المعالجة المتوازية وتوزيع البيانات على أنوية المعالج المتعددة عبر الـ RDD Partitions مع كتابة مباشرة لكل منفذ إلى MongoDB لتجنب أي اختناق في الـ Driver. -

****الملفات الكبيرة (> 200 MB\$)****: يتم توجيهها إلى محرك ****PySpark Loader****

إحصائيات الملف الكبير الرسمية (30 مليون سجل — `orders\_huge\_mixed\_quality.csv`):

التصنيف (Classification) | العدد الدقيق (Exact Count) | النسبة المئوية (Percentage) | المعنى (Meaning)

****إجمالي السجلات الخام (100.00% | 30,209,432**** | ****Total Raw**** | جميع السجلات كما وصلت من المصدر

****السجلات السليمة (75.76% | 22,886,416**** | ****Valid**** | سليمة 100% ولا تحتاج أي تعديل

****السجلات المصححة (14.39% | 4,345,804**** | ****Corrected**** | تم تصحيحها آلياً وتوثيق أثر التعديل (Audit Trail)

****السجلات المعزولة (9.86% | 2,977,212**** | ****Quarantined**** | معزولة لوجود أخطاء جسيمة تمنع المعالجة

****الاتساق الرياضي (Consistency Check)**** | ****PASSED**** | — | $\text{Raw} = \text{Valid} + \text{Corrected} + \text{Quarantine}$

$$\mathbf{30,209,432} = \mathbf{22,886,416} + \mathbf{4,345,804} + \mathbf{2,977,212}$$

2. دليل الملفات والدوال سطرا بسطر

(نقطة الدخول الرئيسية) `main.py` 1

****الوظيفة****: المدخل الرئيسي لتشغيل المشروع عبر سطر الأوامر. -

****الأسطر الهامة****: -

ضبط `sys.stdout.reconfigure(encoding="utf-8")` لمنع انهيار الترميز عند طباعة النصوص العربية في بيئة Windows. -

إضافة المجلد الجذري إلى `sys.path`. -

استدعاء دالة `main` من `src.elt\_pipeline`. -

(الإعدادات المركزية) `config/settings.py` 2

****الوظيفة****: يجمع جميع الثوابت والمتغيرات البيئية ومسارات المجلدات. -

****المتغيرات الأساسية****: -

MongoDB (`mongodb://localhost:27017`) رابط الاتصال بـ: `MONGO\_URI` -

اسم قاعدة البيانات: `DB\_NAME` -

المجموعات الأربع: `orders\_raw`, `orders\_validated`, `orders\_quarantine`, `pipeline\_audit\_logs`. -

الحد الفاصل لتوجيه الملفات: `SMALL\_FILE\_THRESHOLD\_MB = 200` -

حجم دفعة الكتابة الجماعية: `BATCH\_SIZE = 2000` -

(جدول ترجمة الأرقام الهندية/العربية (٩-٠) إلى أرقام لاتينية (9-0): `ARABIC\_DIGITS\_TABLE` -

(\$to 5000\$) "خمسة آلاف", (\$to 2000\$) "قاموس تحويل الكلمات العربية (مثل "ألفان": `ARABIC\_WORDS\_TO\_NUMBERS` -

(`CONFIRMED`, `PAID`, `CASH`, `DELIVERED`, إلخ). قاموس توحيد حالات الطلب والدفع وطرق التوصيل باللغة العربية إلى المعيار الإنجليزي الموحد: `STATUS\_SYNONYMS` -

(موجه الملفات الذكي) `src/file\_router.py` 3

****الكلاس `RouteDecision`****: -

يرجع كائن بيانات يحتوي على: `engine`, `reason`, `size\_mb`, `size\_bytes`, `file\_path`. -

****الدالة `route\_file(file\_path\_str, threshold\_mb, engine\_override)`****: -

تفحص وجود الملف وحجمه الفعلي على القرص عبر `path.stat().st\_size`. -

إذا تم تمرير خيار يدوي (`engine python\_batch` أو `engine pyspark`) تعتمد الخيار اليدوي فوراً مع توضيح السبب. -

إذا لم يتم التمرير اليدوي، تقارن `size\_mb` بالحد `threshold\_mb` (200MB). -

أصغر أو يساوي: تختار `python\_batch` وتبرر ذلك بخفة المحرك وانعدام الـ Overhead. -

أكبر: تختار `pyspark` وتبرر ذلك بالاستفادة من الأنوية المتعددة والتوزيع. -

(إعداد قاعدة البيانات والفهارس) `src/mongo\_setup.py` 4

****الدالة `setup\_collections\_and\_indexes(db)`****: -

تقوم بإنشاء الفهارس الضرورية للسرعة ومنع التكرار: -

1. `orders\_raw`: (للسماح بحفظ تاريخ المحاولات كاملاً Unique Index بدون) `ingested\_at` و `run\_id` فهرس على: -

2. `orders\_validated`: `order\_id` على Unique Index (للمقاومة Idempotency المفتاح التجاري الثابت لضمان الـ) -

3. `orders\_quarantine`: `issue\_key` على Unique Index (لمنع تكرار عزل نفس السجل) -

4. ``pipeline_audit_logs``: Unique Index على ``run_id``.

```
- : **`reset_database(db` الدالة**
```

تسقط المجموعات وتعيد بناء الفهارس من الصفر عند طلب `reset-db` لإجراء اختبارات أداء نظيفة. -

5 `src/quality\_rules.py` (محرك قواعد الجودة والتصنيف)

****الدوال المساعدة**:** -

والمسافات الزائدة، `null`، `NaN`، `None` تنظيف القيمة من: `as\_text(val)`

- `generate\_issue\_key(order\_id, raw\_record)`: إما أن توليد مفتاح فريد لعزل السجل، إما أن تجزئة SHA-256 أو محتوى السجل إذا كان `order_id` مفقودا

****دوال التنظيف الفردية (مع تسجيل أثر التصحيح Audit Trail):** -**

(القواعد 1 و 3 و 4 (الأرقام العربية، الفواصل، الكلمات): `'clean_arabic_and_text_number()'`).

(وإزالة النصوص YER القاعدة 2) استخراج العملة وتوحيدها إلى: `'clean_currency_and_amount()'`

- `clean\_phone\_number()`: 967` (توحيد الهاتف إلى 5 القاعدة XXXXXXXXXX).

- `clean\_email()`: القاعدة 6 (إصلاح الرموز المكررة: `@@` \$to\$ `@`, `..` \$to\$ `.`).

(والتحقق من النطاق 1990-2030 'YYYY-MM-DD' قاعدة ISO 7 تحويل التواريخ إلى صيغة: 'clean_date()').

(القاعدة 8 ترجمة المرادفات العربية وإزالة المسافات: 'clean status and synonyms').

وتفكيك عناصره والتحقق من الكميات والأسعار `items\_json` فحص صحة حقل: `parse\_and\_validate\_items()` -

```
- .**`transform_and_classify_row(raw, run_id` الدالة الرئيسية**`
```

تطبق جميع القواعد وتجمع قائمة `corrections` وقائمة `errors`.

تطبق القاعدة 9** (`TOTAL\_RECONCILED`): إعادة حساب إجمالي الطلب بمطابقة مجموع العناصر + تكلفة التوصيل. -

إذا وجدت أخطاء جوهرية ('CORRUPTED_ITEMS_JSON', 'MISSING_ORDER_ID', إلخ): ترجع ("QUARANTINE", None, quarantine_doc) -

إذا لم توجد أخطاء ولكن وجدت تصحيحات: ترجع ("CORRECTED", validated doc, None) -

إذا كان السجل سليماً 100%: ترجع ("VALID", validated_doc, None).

6 `src/batch\_loader.py` (الدفعي Python محرك)

```
- :.**` (process python batch(file path, db, run id, batch size=2000` الدالة**
```

تفتح الملف بأسلوب 'Streaming' باستخدام 'csv.DictReader' دون تحميل الملف في الذاكرة. -

لكل صف: -

1. تنشئ وثيقة `raw doc` وتضيفها لمصفوفة `raw batch`.

2. تصنف السجل وتجهز عملية `(UpdateOne(..., upsert=True``.

عندما يصل حجم المصفوفة إلى 'batch size': -

```
- `(raw col.insert many(raw batch, ordered=False` تنفيذ
```

- `(quar col.bulk write(quar ops, ordered=False` و `(val col.bulk write(val ops, ordered=False` تنفيذ

تُحسب وتُطبع معدل الإدخال اللحظي 'rows/sec'.

-.(`rows\_read`,`valid\_count`,`corrected\_count`,`quarantine\_count`,`inserted\_count`,`updated\_count`,`unchanged\_count`,`is\_consistent`)
ترجع قاموسا شاملا بالمقاييس والعدادات

7 `src/spark\_loader.py` (الموزع Apache Spark محرك)

```
- :*** (process pyspark batch/file path, db, run id, target partitions=16` الدالة**
```

تُننى `SparkSession` بالوضع المحلي مع استخدام جميع الأنوية `local[\*]` . -

- تستخدم **Fixed Permissive Schema** لقراءة جميع الحقول كـ `StringType` لمنع تشويه البيانات قبل مرحلة التحويل.
- دالة `(partition_worker(partition_rows` :
تعمل داخل كل Spark Executor بشكل مستقل.
- تفتح اتصالاً مباشراً بـ MongoDB وتكتب البيانات بدفعات `bulk_write` مباشرة دون إرسال البيانات الضخمة إلى الـ Driver.
- ترجع للـ Driver فقط ملخصاً صغيراً جداً بالأرقام والعدادات (`yield summary_dict`).
- تضمن هذه التقنية عدم حدوث (Out-Of-Memory (OOM على الـ Driver نهائياً مهما بلغ حجم الملف.

(منسق خط البيانات) `src/elt_pipeline.py` 8

- دالة `(run_pipeline(csv_path, engine_override, threshold_mb, reset_db` :
تفحص وجود الملف، تهيئ MongoDB، تستشير الموجه `file_router`.
- تنفذ المحرك المختار.
- حال فشل تشغيل PySpark (بسبب غياب بيئة جافا أو مشكلة في المنفذ)، يقوم النظام تلقائياً بالتحويل الفوري والذكي إلى `python_batch` وإعادة المعالجة بنجاح دون توقف الخط.
- تحتوي على ميزة **Automated Fallback** في
- تسجل النتائج وتطبع ملخص التنفيذ.

(B محرك التحميل التزايدى - المسار) `src/incremental_loader.py` 9

- دالة `(process_incremental_delta(delta_csv_path, db, run_id` :
تستقبل ملف الـ Delta الذي يحتوي على السجلات الجديدة والمعدلة فقط.
- تطبق قواعد الجودة والتصنيف وتنفذ `Upsert` على `orders_validated`.
- تقيس بدقة:
- السجلات الجديدة المضافة فعلياً: `inserted_count`.
- السجلات التي تم تعديل حالتها أو بياناتها: `updated_count`.
- السجلات المتطابقة تماماً: `unchanged_count`.
- تضمن الـ Idempotency: عند إعادة إرسال نفس ملف الـ Delta، يكون `inserted_count = 0`.

(سكربت الاستعلامات والعرض أمام الدكتور) `src/analytics_queries.py`

- الوظيفة: **بنفذ استعلامات تجميعية متقدمة (Aggregation Pipelines)** على MongoDB لإظهار النتائج الحية للدكتور:
- 1. التحقق من معادلة الاتساق لأخر عملية تشغيل.
- 2. إجمالي المبيعات والإيرادات ومتوسط الطلب حسب المدينة (`'Taiz'`, `'Aden'`, `'Sanaa'`, إلخ).
- 3. أعلى 5 عملاء إنفاقاً.
- 4. تحليل توزيع القواعد التسع التي تم تفعيلها مع عينة لوثيقة مصححة تحتوي `'corrections'`.
- 5. تحليل أسباب العزل مع عينة لوثيقة معزولة.
- 6. توزيع طرق الدفع والتوصيل.

3. قواعد التنظيف التسع والعزل وأثر التصحيح

| القاعدة | رمز القاعدة `rule_code` | المثال الأصلي | القيمة المصححة

1** | الأرقام العربية-الهندية | `'ARABIC_NUMERALS_NORMALIZED'` | `'٥٠٠٠'` أو `'١٢٣'` | `'5000.0'` أو `'123.0'`

2** | توحيد العملة واستخراج السعر | `'5000'` | `'CURRENCY_STANDARDIZED'` ريال أو 'ريال يمني' | `'5000.0'` و العملة `'YER'`

****3**** | فواصل الآلاف | `125,000.00` | `125000.0` | `THOUSANDS\_SEPARATOR\_REMOVED`
****4**** | الأرقام المكتوبة نصاً باللغة العربية | `TEXT\_NUMBER\_CONVERTED` | `الفا` أو `خمسة آلاف` | `2000.0` أو `5000.0`
****5**** | تنظيف وتوحيد أرقام الهواتف | `PHONE\_CLEANED\_DIGITS` | `+967 77 123 4567` أو `...077` | `967771234567`
****6**** | إصلاح الرموز المكررة في البريد | `EMAIL\_REPEATED\_SYMBOLS` | `user@@mail.com` | `user@mail.com`
****7**** | توحيد التواريخ إلى صيغة `31-01-2025` | `DATE\_NORMALIZED\_ISO` | `ISO` أو `2025/01/31` | `31-01-2025`
****8**** | توحيد مرادفات الحالات وطرق الدفع | `STATUS\_SYNONYM\_MAPPED` | `مؤكد`، `نقداً`، `ملغي` | `CONFIRMED`، `CASH`، `CANCELLED`
****9**** | مطابقة وإعادة حساب الإجمالي | `TOTAL\_RECONCILED` | إجمالي غير متطابق مع العناصر والتوصيل | إعادة حسابه بدقة: $\sum(\text{items}) + \text{delivery}$

رموز العزل في `orders\_quarantine`:

- معرف الطلب فارغ أو غير موجود: `MISSING\_ORDER\_ID`
- معرف العميل فارغ: `MISSING\_CUSTOMER\_ID`
- (تاريخ تالف أو خارج النطاق المنطقي) `INVALID\_IMPOSSIBLE\_DATE`: 2030–1990
- للعناصر غير قابل للتحليل أو تالف JSON نص الـ: `CORRUPTED\_ITEMS\_JSON`
- قائمة العناصر فارغة بعد التحليل: `EMPTY\_ITEMS`
- السعر الإجمالي مجهول ولا توجد عناصر لإعادة حسابه: `UNKNOWN\_PRICE`
- (مبالغ مالية سالبة غير مبررة (سعر، توصيل، دفع): `AMBIGUOUS\_NEGATIVE\_VALUE`
- اجتماع أكثر من خطأ جسيم في نفس السجل: `MULTIPLE\_CONFLICTING\_ERRORS`

4. المسار المتقدم B: التحميل التزايدي (Track B)

الخطوات الثلاث لإثبات المسار B:

1. ****التشغيل الأساسي (Initial Load)****:
تحميل العينة الأساسية لتكوين الحالة الابتدائية في `orders\_validated`.
2. ****تشغيل ملف الـ Delta الأول (`delta\_orders.csv`)****:
يحتوي على سجلات جديدة (New Orders) + سجلات معدلة (Modified Orders) + سجلات بدون تعديل (Unchanged).
النتيجة: زيادة عدد السجلات في `orders\_validated` بمقدار السجلات الجديدة فقط (`inserted\_count`)، وتحديث السجلات المعدلة (`updated\_count`).
3. ****إعادة تشغيل نفس ملف الـ Delta مرة ثانية****:
النتيجة: `inserted\_count = 0`، وبقاء عدد السجلات في `orders\_validated` ثابتاً تماماً دون زيادة، مما يثبت الـ ****Idempotency**** رياضياً وعملياً.

5. سيناريوهات "ماذا لو؟" (What-If Analysis)

ماذا لو قام الدكتور بتمرير ملف بمسار مختلف تماماً؟

****الجواب****: النظام مهياً بالكامل لاستقبال أي مسار عبر سطر الأوامر: -

```
python main.py "D:\Doctor_Folder\new_dataset.csv"
```

يقوم الـ Router بقياس حجم الملف الجديد فوراً واختيار المحرك المناسب ومعالجته وتخزينه دون الحاجة لأي تعديل في الكود.

ماذا لو قمنا بتغيير حد الـ Threshold من 200MB إلى 50MB؟

****الجواب****: يتم تغيير الحد إما من خلال متغير البيئة `SMALL\_FILE\_THRESHOLD\_MB` أو عبر سطر الأوامر: -

```
python main.py data/sample_orders_small.csv --threshold 30
```

عندها سيقوم الـ Router بتحويل ملف الـ 41MB إلى محرك `PySpark` بدلاً من `Python Batch`، مما يثبت ديناميكية النظام.

ماذا لو كان حجم الملف 500 مليون سجل (100 جيجابايت)؟

****الجواب****: -

لن تنهار الذاكرة لأننا نستخدم `csv.DictReader` ك Stream مع دفعات محددة (`BATCH\_SIZE=2000`)، وذاكرة الـ RAM تظل ثابتة عند حوالي 80 ميجابايت. -
 في ****Python Batch****:
****PySpark****: المعمارية مصممة باستخدام `mapPartitions` مع اتصالات PyMongo مستقلة لكل Executor، وبالتالي لا يتم سحب البيانات الضخمة إلى الـ Driver نهائياً. -
 في

ماذا لو انقطع الاتصال بقاعدة بيانات MongoDB أثناء المعالجة؟

و (UpdateOne(..., upsert=True` عند إعادة تشغيل الخط بعد عودة الاتصال، يستأنف النظام العمل دون إنشاء أي سجلات مكررة بفضل مفتاح `order_id` الفريد. -
****الجواب****: عمليات الدفعات تستخدم `ordered=False`

6. بنك الأسئلة المتوقعة في المناقشة الشفهية

س1: ما الفرق الجوهرى بين ما قمتم به وبين مشاريع الـ ETL التقليدية؟

أي بيانات، ويسمح بإعادة التحليل مستقبلاً إذا تغيرت القواعد. ثم قمنا بعمليات التحويل والتصنيف داخل الذاكرة ونقل السجلات إلى `orders\_validated` أو `orders\_quarantine`. الخام في `orders\_raw` بياناتها الأصلية مع بيانات التتبع (`engine\_used`, `ingested\_at`, `source\_file`, `run\_id`) قبل أي عملية تنظيف. هذا يضمن عدم فقدان
 < ****الإجابة النموذجية****: في هذا المشروع طبقنا نمط **ELT** الحقيقي؛ حيث قمنا أولاً بتخزين 100% من السجلات

س2: كيف أثبتتم الـ Idempotency عملياً؟

وأثبتنا ذلك بتشغيل نفس الملف مرتين متتاليتين؛ في المرة الأولى تم إدخال السجلات، وفي المرة الثانية كان `inserted\_count = 0` ولم يزداد عدد وثائق المجموعة على الإطلاق.
 وضعنا فهرس فريدة (Unique Index) على حقل المفتاح التجارى الثابت `order\_id` في `orders\_validated`، واستخدمنا عمليات `UpdateOne(..., upsert=True)`. <
****الإجابة النموذجية****:

س3: كيف تجنبتم حدوث (Out-Of-Memory (OOM في PySpark؟

MongoDB وكتابة القسم الخاص به بالتوازي دون إرسال البيانات المجمعة إلى الـ Driver. الـ Driver يستقبل فقط ملخصاً صغيراً جداً بالعدادات عبر `yield summary\_dict`. <
****الإجابة النموذجية****: استخدمنا دالة `mapPartitions` بحيث يقوم كل Executor بإنشاء اتصال مباشر بقاعدة بيانات

س4: كيف نتحقق من عدم فقدان أي سجل؟

< ****الإجابة النموذجية****: نطبق ****معادلة الاتساق الأساسية (Section 6.11)**** في نهاية كل تشغيل:

```
> `rows_read == valid_count + corrected_count + quarantine_count`
```

< ويتم التحقق منها وتسجيلها في تقرير `pipeline\_audit\_logs.is\_consistent`.

7. دليل أوامر التشغيل ورفع المشروع

1. تشغيل الخط على عينة البيانات الصغيرة (Python Batch -> Auto Router):

```
python main.py data/sample_orders_small.csv --reset-db
```

2. تشغيل الخط على أي ملف مخصص يحدده الدكتور:

```
python main.py "مسار_الملف_الجديد" csv
```

3. تشغيل استعلامات المناقشة الشفهية الحية:

```
python src/analytics_queries.py
```

4. تشغيل عرض المسار المتقدم B (التحميل التزايدى و Idempotency):

```
python src/demo_track_b.py
```

5. تشغيل جميع اختبارات الوحدة:

pytest tests/ -v

6. توليد الرسوم البيانية التفاعلية وعرض النتائج:

python src/visualize_results.py

7. رفع المشروع إلى مستودع GitHub الخاص بك:

git remote add origin https://github.com/YOUR_USERNAME/YOUR_REPOSITORY.git

git branch -M main

git push -u origin main

* (ملاحظة: تم ضبط `gitignore` تلقائياً لمنع رفع الملف الضخم 13GB وتجاوز حدود GitHub).

8. المقارنة الشاملة: SQL vs NoSQL (MongoDB) vs Apache Spark

وجه المقارنة | قواعد البيانات العلائقية (SQL / RDBMS) | قواعد بيانات المستندات (NoSQL - MongoDB) | محرك الحوسبة الموزعة (Apache Spark)

المنظمة وإجراء العمليات اليومية (OLTP) | تخزين المستندات شبه المنظمة والمرنة (Document Store) | محرك معالجة وتحليل البيانات الضخمة في الذاكرة (Distributed Compute Engine)
 الدور الأساسي (Primary Role) | تخزين البيانات

| جداول، صفوف، وأعمدة ذات هيكل ثابت (Rigid Schema) | مستندات BSON / JSON مرنة متداخلة (Flexible Schema) | هياكل بيانات موزعة غير قابلة للتغيير (RDDs, DataFrames)
 نموذج البيانات (Data Model)

(Scalability) | توسع رأسي (Vertical Scaling) عبر ترقية مواصفات الخادم | توسع أفقي ممتاز (Horizontal Sharding) وسهل التوزيع | توسع أفقي هائل على مئات الخوادم والأنوية المتعددة
 **قابلية التوسع

Query) مع أوامر `MongoDB Query Language (MQL)` و `JOIN` | Spark SQL, PySpark DataFrame API` و `Aggregation Pipelines`, `RDD` `mapPartitions`
 لغة الاستعلام (SQL) | Structured Query Language (SQL)

عند التعامل مع مئات الجيجابايتات أو الملايين من الصفوف | ممتازة في التخزين والفهرسة ولكن المعالجة الحسابية المعقدة محدودة | الأسرع عالمياً بفضل المعالجة في الذاكرة (In-Memory Computing)
 معالجة البيانات الضخمة (Big Data Processing) | تختنق

تام مع معايير ACID الصارمة | توافق ACID على مستوى المستند الفردي مع Idempotency عبر الفهارس الفريدة | خطط تنفيذ معزولة، ومقاومة للأخطاء (Fault-Tolerant via Lineage Graph)
 الضمانات الرياضية (Guarantees) | توافق

كيفية استخدام والتعامل مع MongoDB في المشروع:

1. رابط الاتصال (**`Connection URI`): `mongodb://localhost:27017`

2. اسم قاعدة البيانات (**`ecommerce\_store`

3. **استعراض البيانات عبر MongoDB Compass

قم بفتح MongoDB Compass والضغط على Connect

اختر قاعدة بيانات `ecommerce\_store` وستشاهد المجموعات الأربع:

- السجلات الأصلية 100% كما وردت من الملف: `orders\_raw`

- السجلات المصححة والسليمة المعتمدة للتحليل: `orders\_validated`

- `quarantine\_reason` السجلات التي تم عزلها مع ذكر سبب العزل في حقل: `orders\_quarantine`

- ملخص تشغيل خط الأنابيب، معدل السرعة، ومعادلة الاتساق: `pipeline\_audit\_logs`

9. الملفات الأربعة الجوهرية في المشروع

إذا سألك الدكتور عن أهم 4 ملفات، يمكنك الإجابة من عدة أبعاد معمارية:

(أ) أهم 4 ملفات كودية (Core Python Modules):

1. المدخل التنفيذي وضابط الواجهة والترميز العربي لسطر الأوامر: `main.py`

2. العقل المدبر لخط الأنابيب الذي يدير التوجيه الذكي ويسجل سجلات التدقيق: `src/elt\_pipeline.py`

3. `src/quality_rules.py`: محرك الجودة والتنظيف الذي ينفذ القواعد التوسع والعزل وتتبع التصحيحات.
4. `src/spark_loader.py`: `mapPartitions` بالتوازي عبر MongoDB ويكتب لـ Partitions الموزع الذي يوزع البيانات على 110 PySpark محرك.

(ب) أهم 4 ملفات بيانات (Datasets):

1. `orders_huge_mixed_quality.csv` (\$12.65\text{ GB}\$ / 30.2 : ملف البيانات الضخمة لاختبار مليون سجل).
2. `data/sample_orders_small.csv` (\$41.77\text{ MB}\$ / 100 : عينة اختبارات الأداء السريعة أمام الدكتور ألف سجل).
3. `data/delta_orders.csv` (\$533\text{ KB}\$): B ملف المسار Idempotency لإثبات التحديث التزاوي والـ B ملف المسار.
4. `أي ملف خارجي جديد يمرره الدكتور` ** لاختبار مرونة وديناميكية النظام.

(ج) المجموعات الأربع في MongoDB:

1. `orders_raw`: وعاء البيانات الخام.
2. `orders_validated`: وعاء البيانات الجاهزة للتحليل التجاري.
3. `orders_quarantine`: مستودع السجلات المعزولة للتدقيق.
4. `pipeline_audit_logs`: سجلات قياس الأداء والاتساق.